

VIRTUELIZACIJA PROCESA

Praktikum
2025/2026

Profesor:
Bojan Jelačić

Asistenti :
Nikola Petrović
Uroš Petrašković

Uvod

Polaganje predmeta :

- Predispitne obaveze – 65 bodova
- Ispitne obaveze – 30 bodova
- Aktivnost 5 bodova

Vežbe :

- Alat koji se koristi prilikom izrade zadataka je Visual Studio (verzija po želji).
- Predispitne obaveze se realizuju u vidu **projektnog zadatka** koji studenti izrađuju u paru (2 člana).
- Da bi student položio predispitne obaveze, potrebno je da ostvari najmanje **35 bodova**, od čega minimalno **5 bodova na eliminacionom zadatku** i **30 bodova** na projektnom zadatku.

Sadržaj

Uvod	1
Sadržaj	2
Vežba 1 - Osnove WCF-a	4
Konfiguracija i pokretanje servisa	5
Rad se složenim podacima	5
Rad sa WCF izuzecima	6
Konfiguracija WCF aplikacija	6
Zadatak 1. WCF	7
Dodatak : Uputstvo za kreiranje projekta	8
Vežba 2 - Manipulacija memorijom	9
Osnovni tipovi podataka	9
Memorijski prostor	9
Dispose pattern	11
Zadatak 1 - Implementacija Dispose patterna	12
Vežba 3 - Rad sa fajlovima	13
Klasa File and FileInfo	13
Klasa Directory i DirectoryInfo	14
Tokovi podataka (Streams)	14
Klase StreamReader i StreamWriter	14
Klasa FileStream	15
Klasa MemoryStream	16
Zadatak 1 - Rad sa direktorijuma i fajlovima	17
Zadatak 2 - File stream	17
Zadatak 3 - Memory stream	17
Vežba 4 - Rad sa fajlovima preko mreže	18
WCF i MemoryStream	18
Zadatak - manipulacija fajlovima preko mreže	18
Vežba 5 – Delegati i događaji	20
Delegati	20
Događaji	21
Deklaracija Događaja	22
Generisanje Događaja	23
Pretplata na Događaj	23
Zadaci	24
Zadatak 1 – Generisanje jednog događaja	24
Zadatak 2 – Generisanje više događaja	24

Vežba 1 - Osnove WCF-a

Windows Communication Foundation (WCF) je framework koji služi za pravljenje servisno-orijentisanih aplikacija. Uz pomoć WCF-a imamo mogućnost slanja asinhronih poruka sa jednog na drugi servis.

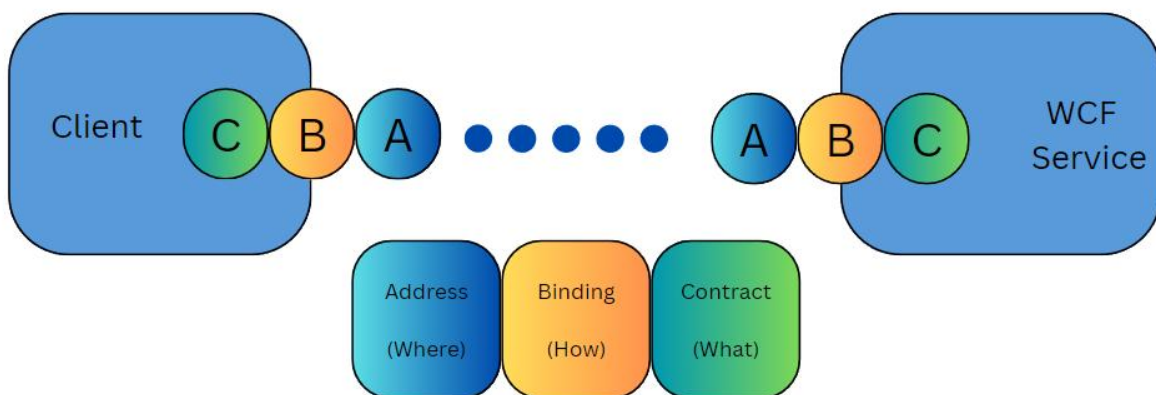
Svaki WCF servis definiše minimalno jedan endpoint koji sadrži skup neophodnih informacija koje su potrebne WCF klijentu za pristup servisu.

Svaki endpoint mora da sadrži sledeće informacije

A. Informacije o adresi na kojoj servis sluša (address) koja je predstavljena u standardnom URI formatu na sledeći način:

Scheme://<MachineName>[:Port]/Path, gde je:

- Scheme: transportni protokol (TCP, HTTP, itd.),
 - Machine name: ime ili adresa računara nam kom je pokrenut (u WCF kontekstu se koristi još i izraz „hostovan“) servis,
 - Port: ovaj parametar je opcioni, i ukoliko se ne navede koristi se podrazumevani broj porta za odgovarajući protokol, npr. 443 za HTTPS.
 - Path: putanja do servisa.
- B. O komunikacionom protokolu (binding),
- C. Informacije o opisu usluge (contract) koje servis pruža klijentima.



Slika 1. Osnovni elementi WCF Endpoint -a

Konfiguracija i pokretanje servisa

Kako bismo pokrenuli neki servis prethodno moramo definisati interfejs koji ćemo koristiti kao Contract. Da bi se jedan interfejs koristio kao contract neophodno ga je dekorisati atributom *ServiceContract*, a metode koje želimo da izlaže sa *OperationContract* atributom (nalaze se u *System.ServiceModel* biblioteci). Primer interfejsa se nalazi ispod.

```
[ServiceContract]
public interface IExample
{
    [OperationContract]
    string TestFunction();
}
```

Rad se složenim podacima

Pored prenošenja ugrađenih tipova WCF nudi mogućnost slanja korisnički definisanih tipova podataka. U WCF-u se koristi *DataContract* serijalizacija prilikom prenosa podataka. Svi ugrađeni .NET primitivni tipovi, kao i neki ugrađeni izvedeni tipovi mogu biti serijalizovani i deserijalizovani pomoću *DataContract* serijalizatora.

Korisnički-definisane tipove podataka je potrebno označiti (dekorisati) sledećim atributima (nalaze se u *System.Runtime.Serialization* prostoru imena iz):

1. *DataContract*: eksplicitno definiše novi tip podatka za serijalizaciju, koristi se za dekorisanje klasa, struktura i enumeracija;
2. *DataMember*: definiše članove klase koji će biti uključeni u proces serijalizacije;
3. *IgnoreDataMember*: navodi se kako bi se preskočila serijalizacija određenog člana klase.

```
[DataContract]
public class Knjiga
{
    public Knjiga(string nazivKnjige, string imeAutora, string prezimeAutora)
    {
        this.NazivKnjige = nazivKnjige;
        this.ImeAutora = imeAutora;
        this.PrezimeAutora = prezimeAutora;
    }

    [DataMember]
    public string NazivKnjige{ get => nazivKnjige; set => nazivKnjige= value; }
    [DataMember]
    public string ImeAutora{ get => imeAutora; set => imeAutora= value; }
    [DataMember]
    public string PrezimeAutora{ get => prezimeAutora; set => prezimeAutora= value; }

    private string imeAutora;
    private string imeAutora;
    private string prezimeAutora;
}
```

Rad sa WCF izuzecima

WCF podržava propagaciju grešaka od servisa do klijenata preko izuzetaka, tj. omogućava da se izuzetak kreira na strani servisa, propagira kroz komunikacioni kanal do klijenta, gde ga klijent može (i treba) uhvatiti sa odgovarajućim kodom za obradu grešaka.

Navođenje izuzetka koji WCF metod može da baci je moguće navođenjem atributa *FaultContract* iznad metode koja može da baci izuzetak.

```
[OperationContract]
[FaultContract(typeof(Exception))]
string TestFunction();
```

Konfiguracija WCF aplikacija

Platforma .NET podržava konfigurisanje izvršnih aplikacija pomoću konfiguracione datoteke koja sadrži konfiguraciju definisanu na XML jeziku. Prevođenjem projekta se pravi kopija ove datoteke sa promenjenim imenom **ime_aplikacije.exe.config**. Konfiguraciju .NET aplikacija je moguće menjati u tim datotekama. Ovakve datoteke se dodaju u projekte koji su izvršnog tipa. Za rad sa konfiguracionim datotekama potrebno je uključiti **System.Configuration** namespace.

U donjem listingu je prikazano na koji način se konfiguriše server.

```
ServiceHost svc = new ServiceHost(typeof(ServiceName));
```

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <system.serviceModel>
    <services>
      <service name="NamespaceOfService.ServiceName">
        <host>
          <baseAddresses>
            <add baseAddress="net.tcp://localhost:4000" />
          </baseAddresses>
        </host>
        <!-- Service Endpoints -->
        <endpoint
          address="Service"
          binding="netTcpBinding"
          contract="NamespaceOfInterface.Interface" />
      </service>
    </services>
  </system.serviceModel>
</configuration>
```

U donjem listingu je prikazano na koji način se konfiguriše klijent.

```
ChannelFactory<InterfaceName> factory =
```

```
new ChannelFactory<InterfaceName>("NazivServisa");
```

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <system.serviceModel>
    <client>
      <endpoint name="NazivServisa"
        address="net.tcp://localhost:4000/Service"
        binding="netTcpBinding"
        contract="Namespace0dInterface.InterfaceName" />
    </client>
  </system.serviceModel>
</configuration>
```

Zadatak 1. WCF

Neophodno je implementirati klijent-server arhitekturu koristeći konfiguracioni fajl prilikom konfigurisanja servisa i klijenta.

Servis treba da izloži sledeće metode :

- Dodavanje novih knjiga u biblioteku
- Brisanje postojećih knjiga iz biblioteke, metoda treba da baci CustomException izuzetak ukoliko pokuša da se obriše knjiga koja ne postoji
- Izlistavanje svih knjiga koje postoje u biblioteci
- Izlistavanje svih knjiga po nazivu autora (ime + prezime)
- Izlistavanje svih knjiga na osnovu zanra
- Izlistavanje svih knjiga koje su izdate određene godine

Treba implementirati klase Knjiga i CustomException

Klasa Knjiga treba da sadrži :

- ID knjige (predstavlja redni broj kreiranog objekta klase)
- Ime knjige
- Ime autora
- Prezime
- Zanr (predstavljen enumeracijom)
- Datum izdavanja
- Za svako odgovarajuće polje obezbediti Property
- Za inicijalizaciju objekta NE koristiti default-ni konstruktor

Klasa CustomException treba da sadrži

- Message i odgovarajući Property

Metode interfejsa implementirati na strani Servisa, sa strane servisa neophodno je da postoji kolekcija Knjiga.

Dodatak : Uputstvo za kreiranje projekta

- Prilikom kreiranja novog projekta neophodno je obratiti pažnju da svi projekti budu **.NETFramework** (Slika 2)



Slika 2. Odabir projekta u VS

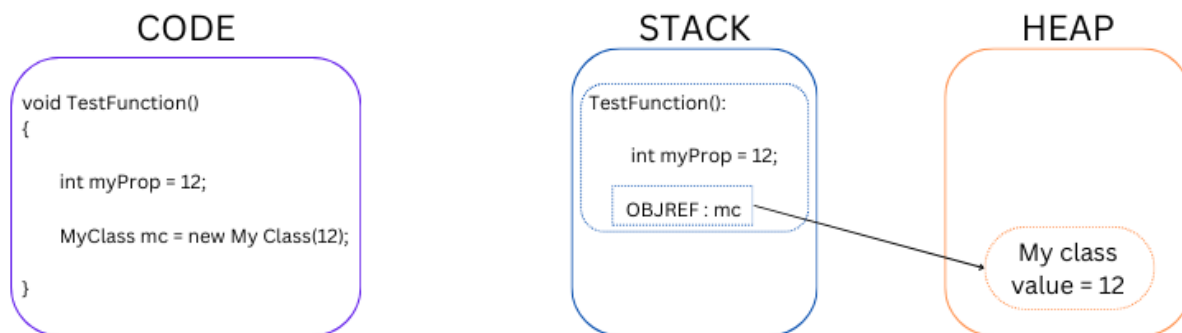
- **Common** projekat je tipa **ClassLibrary**, u njega se piše sve ono što je neophodno da bude vidljivo i klijentu i serveru (interfejsi, klase...)
- Biblioteke **System.ServiceModel** i **System.Runtime.Serialization** u sebi sadrže sve neophodno za kreiranje klijenta (ChannelFactory) i servera (ServiceHost) kao i attribute kojima označavamo ServiceContract i DataContract stoga one moraju biti dodate u one projekte gde je to neophodno
- Servis implementira interfejs i vrši hostovanje kako bi klijent mogao pristupi ovim metodama
- Klijent otvara kanal ka servisu i vrši pozivanje ovih metoda
- Neophodno je dobro konfigurisati app.config kako bi klijent i servis mogli da se povežu uspešno

Vežba 2 - Manipulacija memorijom

Osnovni tipovi podataka

Tip (*type*) je osnovna jedinica programabilnosti u .NET. Postoje dve kategorije tipova podataka:

1. **Vrednosni tipovi (eng. *value types*).** Vrednosni tipovi podataka se izvode iz klase **System.ValueType**. U ovu grupu spadaju: primitivni tipovi podataka (*int, double, bool, char, itd.*), strukture (*struct*) i enumeracije (*enum*). Vrednosti se ovim promenljivama dodeljuju direktno, odnosno svaka promenljiva sadrži direktno dodeljenu vrednost. Ukoliko se prilikom definisanja promenljive ne navede inicijalna vrednost, biće dodeljena podrazumevana vrednost u zavisnosti od tipa. Ovi tipovi podataka se smatraju relativno malim podacima koji se čuvaju direktno na steku(stack) aktivne niti.
2. **Referentni tipovi (eng. *reference types*).** Referentni tipovi podataka ne sadrže vrednost promenljive, već predstavljaju referencu na memorijsku lokaciju na kojoj se promenljiva skladišti. Prema tome, ovaj tip ima dva dela: objekat i referencu na taj objekat. Prilikom dodele instance referentnog tipa drugoj instanci, kopira se referenca instance, ali ne i objekat. Na taj način, više promenljivih ovog tipa mogu da pokazuju na istu memorijsku lokaciju (isti objekat), što implicira da će se izmena vrednosti od strane jedne promenljive reflektovati i na druge promenljive. Promenljive ovog tipa se skladište u okviru posebne memorijske strukture (*heap*) i u potpunosti je kontrolisano od strane *Garbage Collectora* (GC). U referentne tipove spadaju klase, nizova, delegati i interfejsi.



Slika 3. Skladistenja podataka na stack-u i heap-u

Memorijski prostor

Sve podatke vezane za određenu metodu: ulazne parametre, lokalno definisane varijable, adresu linije koda koja treba da se izvrši nakon završetka metode se čuvaju na **Stack**-u. Kada se metoda završi, deo memorije (container) sa vrha se briše, tok izvršavanja se nastavlja na liniji koja je bila zapisana i na vrhu **Stack**-a se opet nalazi metoda koja se trenutno izvršava.

Kada se kreiraju instance referentnog tipa njihova referenca (adresa) se čuva na **Stack**-u dok se sama instanca čuva na Heap-u.

Postoje dve vrste **Heap**-a:

1. Small Object Heap (SOH) za skladištenje objekata veličine manje od 85K
2. Large Object Heap (LOH) za skladištenje objekata veličine veće od 85K

Treba imati u vidu da je skladištenje podataka na **Heap**-u i njihova kontrola od strane **Garbage Collectora** je skupa operacija. Iz tog razloga je objekte koji su relativno mali i kratkog životnog veka (*short-lived*) efikasnije skladištiti na steku niti u okviru koje se kreiraju.

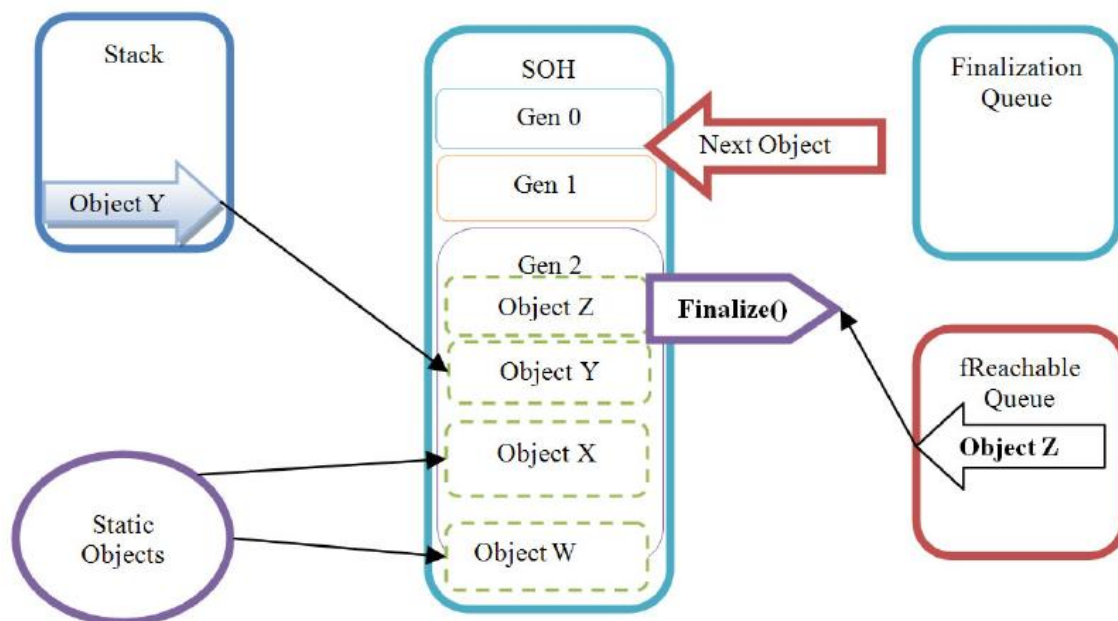
Glavni modul za čišćenje memorije je Garbage Collector (GC). Sve što ovaj modul radi je da prati Heap i registruje sve objekte koji nemaju referencu. Referenca može postojati na Stack-u ali mogu postojati i drugi izvori: globalna promenljiva, CPU registar, referenca na objekat u finalizaciji, interop reference. Sve ove vrste referenci se nazivaju root references ili krace GC roots. Ako neki objekat nema root referencu, praktično ne može biti iskorišćen od strane koda. Postaje beskorisna i zauzeta memorija koja treba da bude počišćena.

GC prikupi sve root reference i krećući se kroz stablo objekata markira sve objekte koji su trenutno u upotrebi (živi objekti). Svi objekti koji nisu markirani treba da budu pokupljeni (collected). Kada se završi markiranje, GC započinje proces čišćenja Heap-ova. Na kraju se resetuje lista objekata koji su još u upotrebi i GC je spreman za novi ciklus.

Ako se piše kod koji koristi neki resurs koji nije managed, recimo rad sa fajlovima, mrežnim resursima, bazama podataka, tada moramo povesti računa da na adekvatan način oslobodimo ove resurse pre nego što se objekat uništi. Ovo se rešava destruktorom ili Finalize metodom. Način na koji se piše destruktor je prikazan u listingu ispod.

```
class TestClass
{
    ~TestClass()
    {
        // code of finalize function
    }
}
```

Svi objekti koji imaju Finalize metodu dobijaju dodatnu root referencu koja se smešta u Finalization queue. Nakon prvog prolaska, kada je objekat spreman za collect, ova referenca se briše iz Finalization queue i premešta u fReachable queue. U ovom slučaju je ovo i jedina referenca i Finalize metoda će biti izvršena. Finalize metoda na ovaj način produžava vek objektu i samim tim nepotrebno zauzima memoriju. Ako imamo veliki broj objekata koji se često alociraju i dealociraju i koji su pri tom veliki, možemo lako doći u situaciju da imamo veliki deo memorije zauzet a da zapravo naša aplikacije ne koristi aktivno tu memoriju.



Slika 4. Zauzimanje i oslobađanje resursa od strane GC

Dispose pattern

Da bi se unapredio ovaj mehanizam dealociranja objekata sa Heap-a koji imaju u sebi unmanaged resurse, predložen je efikasniji način čišćenja resursa. Dispose pattern predlaže da se za sve klase implementira Dispose metoda koja će počistiti alocirane unmanaged resurse i u isto vreme briše referencu iz Finalization queue. Metoda se poziva ručno na mestu gde objekat više nije potreban i gde resursi mogu biti oslobođeni.

```
public void Dispose()
{
    Cleanup(true);
    GC.SuppressFinalize(this);
}

private void Cleanup(bool disposing)
{
    if (disposing)
    {
        // Thread-specific code goes here
    }
}
```

Klasa u kojoj se implementira Dispose pattern bi trebala da implementira interfejs IDisposable. Pravilna implementacija Dispose pattern je prikazana u listingu ispod.

```

public class UnmanagedWrapper : IDisposable
{
    private bool disposed = false;
    public UnmanagedWrapper()
    {
        // creates the unmanaged resource...
    }
    ~UnmanagedWrapper()
    {
        Dispose(false);
    }
    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);
    }
    protected virtual void Dispose(bool disposing)
    {
        if (!disposed)
        {
            // Free the unmanaged resource anytime.
            if (disposing)
            {
                // Free any other managed objects here.
            }
            disposed = true;
        }
    }
}

```

Dispose pattern je praktično obavezan alat u radu sa unmanaged resursima. Finalize metodu treba implementirati na način da se poziva Cleanup jer će na taj način resursi biti počišćeni iako se u kodu izostavi eksplicitno pozivanje Dispose metode (dodatna sigurnost).

Zadatak 1 - Implementacija Dispose patterna

Napraviti klasu koja implementira IDisposable interfejs i Dispose pattern i služi za manipulaciju tekstualnim fajlovima.

Ova klasa treba da ima polje putanja, kao i polja za TextWriter i TextReader koja će posle biti oslobođena prilikom implementacije Dispose patterna. Klasa poseduje konstruktora koji sadrži parametar putanju, ova putanja ne može da se menja tokom zivota jednog objekta, ali u svakom momentu može da se dođe do informacije koja je njena vrednost.

Ova klasa treba da ima metode za dodavanje teksta na kraj fajla, brisanje celog teksta iz fajla, funkciju za čitanje celog teksta iz fajla i metodu koja prima neograničen broj parametara i broji pojave u tekstu, pod pojavama se podrazumevaju da se prosledjeni parametri poklapaju sa celom rečju.

U okviru klase Program.cs napraviti meni koji nudi korisniku mogućnost da vrši manipulaciju sa fajlom čije ime navede sa mogućnošću poziva svih metoda koje su gore navedene, kao i mogućnost da završi manipulaciju nad fajlom, prilikom završetka manipulacije nad određenim fajlom neophodno je pozvati metodu za čišćenje resurs i omogućiti korisniku ponovno unošenje putanje ukoliko želi da radi nad drugim fajlom ili završetak programa. Svaku od metoda pozivati u okviru try-catch bloka.

Vežba 3 - Rad sa fajlovima

Biblioteka koja sadrži sve neophodno za manipulaciju nad fajlovima i strimovima (čitanje, pisanje) je *System.IO Namespace*.

Klasa File and FileInfo

Klasa *File* pruža statičke metode za kreiranje, kopiranje, brisanje, premeštanje i otvaranje datoteka. Pomaže u kreiranju *FileStream* objekata. Ova klasa se koristi ukoliko je neophodno da izvršite neku operaciju nad fajlom u datom momentu, pored toga pruža mogućnost izmene informacija o samom datumu kreiranja fajla.

Neke od metode su:

- **ReadAllText** - otvara tekstualnu datoteku i celokupan tekst iz nje učitava u string
- **ReadAllLines** - otvara tekstualnu datoteku i iz nje učitava sve redove u niz stringova.
- **ReadAllBytes** - otvara binarnu datoteku i iz nje učitava sadržaj u niz bajtova
- **ReadLines** - čita pojedinačne redove tekstualne datoteke i upisuje ih u niz stringova. Metoda pogodna za velike tekstualne datoteke.
- **WriteAllText** - kreira novu tekstualnu datoteku i upisuje u nju tekstualni sadržaj.
- **WriteAllLines** - kreira novu datoteku i upisuje u nju jedan ili više stringova
- **WriteAllBytes** - kreira novu binarnu datoteku i upisuje u nju prosleđeni niz bajtova
- **AppendAllText** - otvara datoteku i dodaje tekst na kraj datoteke
- **AppendAllLines** - ovara datoteku i dodaje sve redove na kraj datoteke
- **Open** - otvara *FileStream* na specifičnoj putanju
- **Copy** - kopira postojeći fajl u novi fajl
- **Create** - kreira ili ukoliko fajl postoji prepíše ga
- **Delete** - briše fajl

Klasa *FileInfo* pruža informacije o datotekama. Omogućava kreiranje, kopiranje, brisanje, premeštanje i otvaranje datoteka. Podržava kreiranje *FileStream* objekata.

Neke od metoda su:

- **AppendText** - kreira *StreamWriter* koji dodaje tekst u fajl
- **Open** - otvara fajl
- **CopyTo** - kopira postojeći fajl u novi fajl
- **Create** - kreira fajl
- **Delete** - trajno briše fajl

Klasa *File* se koristi kada je neophodno izvršavati samo jednu operaciju nad fajlom, dok se klasa *FileInfo* koristi ukoliko treba da se izvršava više operacija. Još jedna od osnovnih razlika jeste ta da kada želimo da koristimo klasu *FileInfo* neophodno je da napravimo objekat te klase i onda nad tim objektom da pozivamo metode, dok je klasa *File* statička i ne postoji mogućnost kreiranja njene instance.

Klasa Directory i DirectoryInfo

Klasa *Directory* sadrži statičke metode za kreiranje, premeštanje i prolazak kroz direktorijume i poddirektorijume.

Neke od metoda su :

- **CreateDirectory** - kreira direktorijum sa poddirektorijumima
- **Delete** - briše direktorijum
- **Exists** - vrši proveru da li direktorijum postoji
- **Move** - vrši pomeranje direktorijuma sa jedne destinacije na drugu
- **GetDirectories** - vraća informaciju o imenima direktorijuma kao i njihovim putanjama
- **GetFiles** - vraća informaciju o nazivima fajlova kao i njihovim putanjama

Klasa *DirectoryInfo* pruža informacije o direktorijumima. Omogućava kreiranje, premeštanje i prolazak kroz direktorijume i poddirektorijume.

Neke od metoda su:

- **Create** - kreira direktorijum sa poddirektorijumima
- **Delete** - briše direktorijum
- **Exists** - vrši proveru da li direktorijum postoji
- **MoveTo** - vrši pomeranje direktorijuma sa jedne destinacije na drugu
- **GetDirectories** - vraća informaciju o imenima direktorijuma kao i njihovim putanjama
- **GetFiles** - vraća informaciju o nazivima fajlova kao i njihovim putanjama

Razlika između ove dve klase je u tome što *DirectoryInfo* zahteva kreiranje objekta nad kojim će se vršiti pozivanje metoda dok *Directory* predstavlja statičku klasu koja ne može da se instancira. Ukoliko nad nekim resursom treba da se izvršavaju različite operacije više puta preporuka je da se koristi *DirectoryInfo*.

Tokovi podataka (Streams)

Stream-ovi su nizovi bajtova koji se mogu koristiti za upisivanje i čitanje iz različitih vrsta skladišta podataka.

Apstraktna klasa *Stream* podržava čitanje i pisanje bajtova. Sve klase koje implementiraju stream-ove nasleđuju klasu *Stream*. Klasa *Stream* i njene izvedene klase pružaju zajednički pogled na izvore podataka i spremišta. Klasa *Stream* i njene izvedene klase apstrahuju detalje operativnog sistema i hardware-a. Prednosti stream-ova je u inkrementalnom procesiranju podataka, apstrakcija rada sa skladištima podataka kao i fleksibilnosti.

Klasa *Stream* implementira *IDisposable* interfejs. Pošto koristi eksterne resurse, stream se čisti garbage collector-om, te ga je potrebno ručno počistiti korišćenjem *Dispose()* metode.

Klase StreamReader i StreamWriter

Klasa *StreamWriter* nasleđuje apstraktnu klasu *TextWriter* služi za upisivanje karaktera u Stream u određenom enkodingu.

Neke od metoda su :

- **Write** - Upisuje string u stream
- **WriteLine** - Upisuje string u stream i dodaje karakter za novi red na kraju
- **Close** - Zatvara stream
- **Dispose** - Oslobađa se resursa

Klasa *StreamReader* nasleđuje apstraktnu klasu *TextReader* i služi za čitanje karaktera iz *Stream*-a u odredjenom enkodingu.

Neke od metoda su:

- **Peek** - Vraca prvi sledeći karakter
- **Read** - Čita sledeći karakter u stream-u
- **ReadLine** - Čita sledeću liniju i vraća je kao string
- **ReadToEnd** - čita karaktere od određene pozicije do kraja

Klasa FileStream

FileStream klasa implementira stream za datoteku, podržavajući operacije čitanja i pisanja u datoteku i iz datoteke.

Neke od metoda su :

- **BeginRead** - operacija čitanja u asinhronom režimu
- **BeginWrite** - operacija pisanja u asinhronom režimu
- **Read** - metoda pomoću koje se vrši čitanje niza bajtova iz fajla
- **Write** - metoda pomoću koje se vrši pisanje niza bajtova u fajl
- **Close** - metoda koja zatvara stream

U donjem listingu je dat primeri kako se koristi *FileStream* prilikom čitanja i pisanja bajtova u fajl.

```
using (FileStream fs = File.Create(path))
{
    string line = "Some text for writing in file";
    byte[] lineInBytes = new UTF8Encoding(true).GetBytes(line);
    fs.Write(lineInBytes, 0, lineInBytes.Length);
}

using (FileStream fs = File.OpenRead(path))
{
    byte[] buffer = new byte[1024];
    UTF8Encoding temp = new UTF8Encoding(true);
    fs.Read(buffer, 0, buffer.Length);
    temp.GetString(buffer, 0, buffer.Length);
}
```


Klasa *MemoryStream*

MemoryStream implementira stream za memoriju, podržavajući operacije čitanja i pisanja u memoriju i iz memorije.

Neke od metoda su :

- **BeginRead** - operacija čitanja u asinhronom režimu
- **BeginWrite** - operacija pisanja u asinhronom režimu
- **Read**- metoda pomoću koje se vrši čitanje iz fajla
- **Seek** - metoda sa kojom se vrši pozicioniranje na određeni deo fajla
- **WriteTo** - metoda pomoću koje se vrši pisanje u fajl
- **Close** - metoda koja zatvara stream

U donjem listingu je dat primer upisa podataka bajt po bajt u *MemoryStream*, kao i samo čitanje iz *MemoryStream*-a.

```
UnicodeEncoding uniEncoding = new UnicodeEncoding();
byte[] textInBytes = uniEncoding.GetBytes(text);

using (MemoryStream memStream = new MemoryStream())
{
    // Write string to the stream byte by byte.
    int count = 0;
    while (count < textInBytes.Length)
    {
        memStream.WriteByte(textInBytes[count++]);
    }
}

using (MemoryStream memStream = new MemoryStream())
{
    byte[] byteArray = new byte[memStream.Length];
    int count = memStream.Read(byteArray, 0, byteArray.Length);
    char[] charArray = new char[uniEncoding.GetCharCount(byteArray, 0, count)];
    uniEncoding.GetDecoder().GetChars(byteArray, 0, count, charArray, 0);
}
```


Zadatak 1 - Rad sa direktorijuma i fajlovima

Napraviti program koji služi za manipulaciju sa direktorijumima i fajlovima i ima sledeće metode :

- **AddDirectory(string path)** - metoda za dodavanje direktorijuma
- **RemoveDirectory(string path)** - metoda za uklanjanje direktorijuma
- **GetListOfAllFiles(string path)** - metoda koja vraća spisak svih fajlova u određenom direktorijumu
- **CreateEmptyFiles(string directoryPath, params string[] fileNames)** - metoda koja kreira fajlove na određenoj putanji, pomoću ove metode može da se kreira neograničen broj fajlova
- **RemoveFile(string path)** - metoda koja uklanja fajl
- **ReadFile(string path)** - metoda koja vraća ceo sadržaj fajla (koristiti klasu StreamReader)
- **AddTextToFile(string path, string text)** - metoda koja dodaje tekst u fajl (koristiti klasu StreamWriter)

Sve ove metode testirati u okviru klase Program.cs

Zadatak 2 - File stream

U okviru Programa.cs napraviti funkcionalnost čitanja i pisanja u fajl koristeći *FileStream* klasu. Na početku pokretanja programa korisnik treba da unese putanju fajla koji treba da kreira, nakon toga unosi tekst koji želi da se nađe u fajlu. Mogućnost završetka unošenja teksta se korisniku omogućavata na taj način da unese END u novi red. Kada je korisnik uneo sav tekst neophodno je na konzolu ispisati sav tekst koji se nalazi u fajlu.

Zadatak 3 - Memory stream

U okviru Programa.cs koristeći *MemoryStream* klasu serijalizovati teks koji je korisnik uneo na konzolu i upisivati ga bajt po bajt u objekat *MemoryStream*-a, potom serijalizovati niz bajtova u string i ispisati ga na konzolu. Sve ovo uraditi u jednom *using*-u odnosno kreirati samo jedan objekat *MemoryStream*-a.

Vežba 4 - Rad sa fajlovima preko mreže

Sistem za upravljanje bazom podataka (DBMS) predstavlja softverski alat koji korisnicima omogućava da lako upravljaju bazom podataka. Ovakvi sistemi imaju mogućnost čuvanja datoteka u tabelama, najčešće korišćenjem BLOB (binary large object) tipova. Ovakvi objekti mogu biti do reda veličine 2GB.

Prilikom slanja podataka preko mreže neophodno je fajl serijalizovati i tako ga preneti preko mreže. `MemoryStream` klasa pruža mogućnost konverzije fajlova u bajtove i obrnuto.

WCF i `MemoryStream`

Parametri koji se prosleđuju kroz WCF i sadrže `MemoryStream` objekte treba sami da implementiraju `IDisposable` interfejs, gde `Dispose` metoda treba da se oslobodi svih resursa koji se više ne koriste.

Kada neka od metoda interfejsa sadrži parametar koji implementira `IDisposable` interfejs preko atributa `OperationBehaviour` možemo da naglasimo da li će resurse koji se prosleđuju kroz parametre oslobađati server. Ukoliko je vrednost podešena na `true`, što je i podrazumevana vrednost onda računamo da hoće. Ovaj atribut se dodaje iznad implementacije metode interfejsa.

```
[OperationBehavior(AutoDisposeParameters = true)]
```

Zadatak - manipulacija fajlovima preko mreže

Neophodno je napraviti klijent-server arhitekturu.

Deljeni interfejs treba da sadrži metode za slanje fajla i za dobavljanje fajlova.

```
FileManipulationResults SendFile(FileManipulationOptions options);  
FileManipulationResults GetFiles(FileManipulationOptions options);
```

U okviru deljene biblioteke treba da se nađu i klase `FileManipulationResults` i `FileManipulationOptions`, obe ove klase treba da implementiraju interface `IDisposable`. Klasa `FileManipulationOptions` sadrži polja `memoryStream` (tipa `MemoryStream`) i `keyWord` (tipa `string`). Klasa `FileManipulationResults` sadrži polja `resultType` (tipa `enum ResultType` koja može da ima vrednosti `Success`, `Warning`, `Failed`), `resultMessage` (tipa `string`) i `memoryStreamCollection` (tipa `Dictionary<string, MemoryStream>`).

U listingu dole je prikazano na koji način se definiše varijabla u konfiguracionom fajlu.

```
<appSettings>  
  <add key="path" value="Files"/>
```

```
</appSettings>
```

U listingu dole je prikazano na koji način se pristupa varijabli iz koda.

```
var fileDirectoryPath = ConfigurationManager.AppSettings["path"];
```

Metode interfejsa je neophodno impelentirati sa strane sarvisa tako da:

SendFile - Na putanju koja je definisana u konfiguracionom fajlu treba upisati prosleđen fajl, treba voditi računa da ukoliko putanja nije definisana ili je prosleđen prazan fajl servisa vraća upozorenje sa porukom, ukoliko se uspešno sačuva fajl šalje se informacija da je fajl uspešno sačuvan, dok ukoliko je došlo do neke greške vraća se informacija da operacija nije uspešno izvršena.

GetFiles - Za putanju koja je definisana u konfiguracionom fajlu treba isčitati sve fajlove koji počinju sa ključnom rečju koja je prosleđena kroz parametar, treba voditi računa da ukoliko putanja nije definisana servis vraća upozorenje sa porukom, ukoliko se uspešno isčitaju fajlovi šalje se informacija da je operacija uspešno izvršena, dok ukoliko je došlo do neke greške vraća se informacija da operacija nije uspešno izvršena.

Sa strane klijenta neophodno je napraviti meni koji nudi mogućnost pozivanja metoda servisa, neophodno je obraditi izuzetak ukoliko se desi, prilikom vraćanja fajlova sa servisa neophodno je iste upisati u direktorijum.

Vežba 5 – Delegati i događaji

Delegati (engl. *delegates*) i događaji (engl. *events*) su koncepti u C# programiranju koji doprinose efikasnom upravljanju događajima i asinhronim operacijama u aplikacijama. Kombinacija delegata i događaja je pogodna za razvoj aplikacija sa korisničkim interfejsom koje efikasno odgovaraju na korisničke akcije i na promene u sistemu.

Delegati

Delegat predstavlja pokazivač na metode koji omogućava dinamičko pozivanje tih metoda na osnovu unapred definisanog potpisa. Implementacija odgovarajućih metoda nije unapred određena, već se prepušta objektu ili metodi koji koriste delegat. Korišćenje delegata omogućava implementaciju *callback* funkcija (funkcija koje se mogu proslediti drugim funkcijama kao argumenti) što znači da program može pozvati određenu funkciju ili metodu u odgovarajućem trenutku tokom izvršavanja, na osnovu nekog događaja ili uslova.

Delegati se deklariraju korišćenjem ključne reči **delegate**, pored čega je potrebno navesti potpis metode koji će delegat referencirati. Potpis metode uključuje tip povratne vrednosti i tipove parametara koje metoda prima. Kada se delegat deklarise, instanca delegata će pozivati metode čiji tip povratne vrednosti i lista parametara odgovaraju deklaraciji delegata.

delegate [*return_type*] [*delegate_name*] ([*parameter_list*])

U donjem listingu je dat primer deklaracije delegata *MathOperation* koji omogućava pozivanje metoda *Add* i *Subtract*.

```
public delegate int MathOperation(int x, int y);
public class Example
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Subtract(int a, int b)
    {
        return a - b;
    }

    private int Calculate(MathOperation mathOperation, int a, int b)
    {
        return mathOperation(a, b);
    }

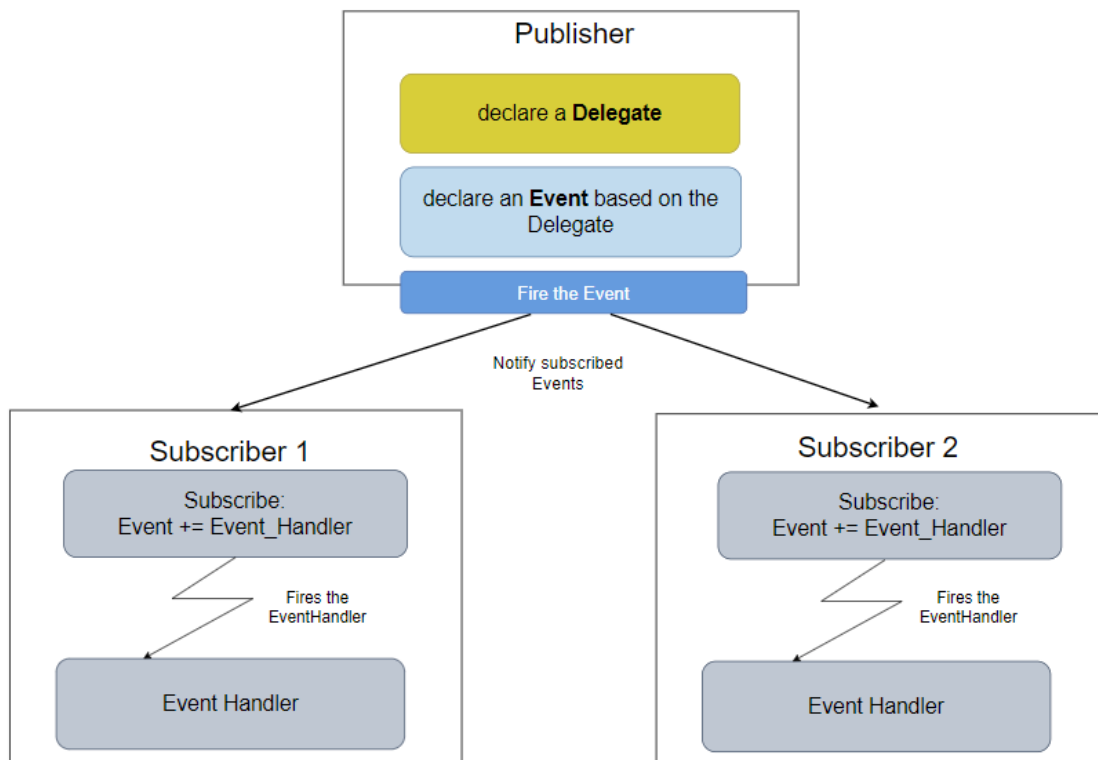
    public void Test()
    {
        int test1 = Calculate(Add, 10, 5);
        int test2 = Calculate(Subtract, 10, 5);
    }
}
```

Događaji

Događaj u C#-u predstavlja mehanizam komunikacije između različitih delova softvera ili aplikacije. Ovaj mehanizam omogućava objektu da obavestiti druge delove softvera o nekoj akciji ili promeni koja se dogodila. Često se koriste u grafičkim korisničkim interfejsima i asinhronom programiranju za upravljanje korisničkim akcijama, kao što su klikovi miša ili pritisci na tastaturi. Događaji mogu biti i različite sistemske notifikacije na koje aplikacija treba da odreaguje, kao što su promene u mrežnom statusu ili dolazak novih podataka. Korišćenjem događaja, objekti mogu efikasno komunicirati i reagovati na promene u okruženju bez potrebe za neprekidnom proverom stanja. Za implementaciju događaja su potrebne:

- Klasa koja generiše događaj (tzv. generator događaja, *publisher*) i
- Klasa koja želi da bude obavještena o nekom događaju (potrošač, *subscriber*)

Mehanizam događaja koristi *publish-subscribe* softverski obrazac (engl. *design pattern*). Izdavač (*publisher*) je objekat koji sadrži definiciju događaja i delegat koji implementira rukovaoca događajem (*event handler*). *Publisher* klasa generiše događaj i obaveštava zainteresovane objekte. Pretplatnik (*subscriber*) je objekat koji prihvata događaj i implementira rukovanje događajem. Događaji se definišu delegatom. Zainteresovane komponente se registruju na događaj dodavanjem svojih *event handler*-a. Ukoliko komponenta više nije zainteresovana za događaj, slično dodavanju, može da ukloni svoj *event handler*. Grafički prikaz opisanog mehanizma je dat na slici 5.



Slika 5. Grafički prikaz mehanizma za generisanje događaja

Primer: Klasa koja implementira kontrolu korisničkog interfejsa može definisati događaj koji se pokreće kada korisnik napravi levi klik mišem. Klasu ne zanima šta će se dogoditi nakon klika korisnika, ali je potrebno da obavesti druge klase (pretplatnike) da se klik događaj desio. Klase koje su zainteresovane za događaj odnosno tzv. pretplatnici, treba da implementiraju rukovanje događajem.

Deklaracija Događaja

Događaji se deklariraju u okviru klase koja će generisati te događaje. Uz događaj moguće je generisati i informacije o samom događaju (*event arguments*) i poslati ih stranama zainteresovanim za taj događaj. Da bi se događaj definisao, potrebno je prvo definisati delegat koji predstavlja metode koje će se pozivati kada se događaj desi, a zatim i sam događaj. Na ovaj način, u donjem listingu, definisan je delegat pod nazivom *MyEventHandler* i događaj *MyEvent*, koji kada se generiše poziva delegat *MyEventHandler*.

```
public class Publisher
{
    public delegate void MyEventHandler(string message);
    public event MyEventHandler MyEvent;
}
```

Preporučeni *Microsoft* obrazac za definisanje događaja naveden je u listingu ispod.

```
public class Publisher
{
    public delegate void MyEventHandler(object sender, EventArgs e);
    public event MyEventHandler MyEvent;
}
```

Prvi parametar je tipa *object* i predstavlja referencu na objekat koji je generisao događaj. Drugi parametar je tipa *EventArgs* koji sadrži niz informacija o događaju. Ukoliko je potrebno poslati dodatne, korisnički definisane informacije o generisanom događaju (npr. prilikom njegovog generisanja), onda je moguće kreirati i klasu koja nasleđuje *EventArgs* koja će sadržati dodatne informacije.

U donjem listingu je dat primer upotrebe klase koja nasleđuje *EventArgs*.

```
public class Publisher
{
    public delegate void MyEventHandler(object sender, CustomEventArgs e);
    public event MyEventHandler<CustomEventArgs> MyEvent;
}

public class CustomEventArgs : EventArgs
{
    public string EventMessage { get; }

    public CustomEventArgs(string eventMessage)
    {
        EventMessage = eventMessage;
    }
}
```

Generisanje Događaja

Prilikom generisanja događaja potrebno je proveriti da li je događaj različit od *null*, odnosno proverava se da li ima registrovanih klijenata na dati događaj. Ukoliko ih ima, potrebno je generisati („okinuti“) taj događaj, navodeći naziv događaja zajedno sa parametrima delegata na osnovu kog je događaj definisan.

U donjem listingu je dat primer generisanja događaja.

```
public void RaiseEvent(string message)
{
    Console.WriteLine("Something happend...");

    if(MyEvent != null)
    {
        MyEvent(this, new CustomEventArgs(message));
    }
}
```

Pretplata na Događaj

Za pretplate (*subscribe*) i odjave (*unsubscribe*) na događaje u C# se koriste sledeće operacije:

- **+=** dodaje instancu delegata u listu poziva, na ovaj način handler će biti pozvan kada se događaj desi
- **-=** koristi se za uklanjanje handlera za događaj. Otkazivanje pretplate na događaje je važan korak za upravljanje memorijom.
-

U donjem listingu je dat primer pretplate klase *EventSubscriber* na događaj *MyEvent*. Kada se ovaj događaj desi, pozvaće se metoda *OnEventRaised*.

```

public class EventSubscriber
{
    public EventSubscriber(Publisher publisher)
    {
        publisher.MyEvent += OnEventRaised;
    }

    private void OnEventRaised(object sender, CustomEventArgs e)
    {
        Console.WriteLine($"React to an event!
                           Received message:{e.EventMessage}");
    }
}

```

Zadaci

Zadatak 1 – Generisanje jednog događaja

Napisati program koji, koristeći *publish-subscribe* obrazac, implementira sledeće dve komponente:

- 1) **AlarmGenerator** koji generiše alarm svake 3 sekunde,
- 2) **Listener** koji prilikom svakog generisanog alarma ispisuje poruku "Heard it!" zajedno sa vremenskim trenutkom kad je alarm izgenerisan.

Posle petog oglašavanja alarm prestaje sa radom. Broj oglašavanja alarma treba pročitati iz konfiguracionog fajla. Događaj je potrebno definisati koristeći šablon preporučen od strane kompanije *Microsoft*.

Zadatak 2 – Generisanje više događaja

Napisati zadatak koji, koristeći *publish-subscribe* obrazac, implementira sledeće komponente:

- 1) **TransactionGenerator** koji služi za upravljanje novčanim transakcijama. Iznos transakcije može biti pozitivan ili negativan broj. Potrebno je generisati dva događaja. Prvi događaj se generiše ukoliko je rezultat transakcija uvećanje iznosa na računu dok drugi događaj označava da je rezultat transakcije smanjenje iznosa na računu. Na početku programa, potrebno je da korisnik izabere tip poruke putem koje će dobiti obaveštenje o izvršenoj transakciji.
- 2) **TransactionTextObserver** predstavlja komponentu koja reaguje na događaj tako što u konzoli ispiše poruku da je obaveštenje stiglo putem *sms* poruke.
- 3) **TransactionEmailObserver** predstavlja komponentu koja reaguje na događaj tako što u konzoli ispiše poruku da je obaveštenje stiglo putem *email* poruke.

Prilikom generisanja događaja potrebno je ispisati informaciju o iznosu transakcije, i da li je došlo do povećanja ili smanjenja stanja na računu. Dodatne informacije o događaju mogu se preneti pomoću klase **PaymnetEventArgs** koja sadrži polja *Amount* (int) koje predstavlja količinu novca i *Message* (string) koja označava da li je došlo do povećanja ili umanjavanja stanja na računu.

